

DOLLOR.AI

Complete System Architecture

Customer App • Driver App • Restaurant App • Rideshare

December 12, 2024

Platform Status: PRODUCTION READY

TABLE OF CONTENTS

1. System Overview
2. Complete Architecture Diagram
3. Customer App Architecture
4. Driver App Architecture
5. Restaurant App Architecture
6. Rideshare System Architecture
7. API Connection Details
8. Database Schema
9. Payment Flow
10. Real-time Communication

1. SYSTEM OVERVIEW

Dollor.AI is a P2P food delivery and rideshare platform with \$1-\$3 tiered pricing.

Component	Technology	Purpose
Customer App	SwiftUI (iOS 17+)	Food ordering & rideshare booking
Driver App	SwiftUI (iOS 17+)	Delivery & ride fulfillment
Restaurant App	SwiftUI (iOS 17+)	Order & menu management
Backend API	FastAPI (Python)	136 REST endpoints
Database	PostgreSQL (AWS RDS)	Data persistence
Payments	Stripe Connect	P2P payments
Auth	JWT + Firebase	Authentication
Maps	Google Maps SDK	Location & routing
Push	Firebase FCM	Notifications

2. COMPLETE ARCHITECTURE DIAGRAM

How all components connect:

[illegible]

3. CUSTOMER APP ARCHITECTURE

Bundle ID: com.dollor.customer

[illegible]

4. DRIVER APP ARCHITECTURE

Bundle ID: com.dollor.driver

[illegible]

5. RESTAURANT APP ARCHITECTURE

Bundle ID: com.dollor.restaurant

```

##### RESTAURANT APP FLOW #####
##### 
##### USER INTERFACE #####
##### VendorLoginView /
RestaurantDashboardView ##### OrdersView (Incoming orders, accept/reject) ##### MenuManagementView (Add/edit menu items) #####
##### MarkItemsUnavailableView (86 items) ##### PromotionsView (Create deals) ##### AnalyticsView (Sales, performance) #####
##### SettingsView (Store hours, profile) #####
##### ▼ #####
##### VIEW MODELS #####
##### VendorAuthManager ##### OrdersViewModel
MenuViewModel ##### • login() • fetchOrders() • fetchMenu() ##### • register() • acceptOrder() • addItem() ##### •
updateProfile() • rejectOrder() • updateItem() ##### • markReady() • deleteItem() #####
##### ▼ #####
##### SERVICES #####
##### P2PAPIService.swift (Shared API
client) ##### NotificationManager (Order alerts via FCM) #####
##### ▼ #####
##### API ENDPOINTS USED #####
AUTH: ##### POST
/api/auth/vendor/login (form-data) ##### POST /api/auth/vendor/register ##### POST /api/auth/vendor/google-auth ##### POST
/api/auth/vendor/apple-auth ##### ORDERS: GET /api/erp/orders/vendor/{id} ##### GET /api/vendors/{id}/orders #####
POST /api/erp/orders/{id}/accept ##### POST /api/erp/orders/{id}/reject ##### POST /api/erp/orders/{id}/ready #####
MENU: GET /api/vendors/{id}/menu ##### GET /api/vendors/{id}/menu/categories ##### POST /api/vendors/{id}/menu ##### PUT
/api/vendors/{id}/menu/{item_id} ##### DELETE /api/vendors/{id}/menu/{item_id} ##### PROMOTIONS: GET
/api/promotions/vendor/{id} ##### POST /api/promotions/vendor/{id} ##### GET /api/promotions/analytics/{id} ##### GET
/api/promotions/suggestions/{id} #####

```

6. RIDESHARE SYSTEM ARCHITECTURE

Complete rideshare flow from request to completion:

```

RIDESHARE SYSTEM ARCHITECTURE
RIDESHARE APP BACKEND DRIVER APP
TripBoardView AvailableRides View
Pickup loc Drop loc See nearby Fare est Accept ride
1. Request Ride POST /api/rides/request {pickup, dropoff, customer_id}
BACKEND PROCESSING
Base fare: $2.50 Per mile: $1.50 Platform fee: $1-3 (tiered) 1. Calculate fare based on distance
Query drivers table WHERE is_online = true 2. Find nearby available drivers (within 5 miles)
Filter by driver_type IN ('rideshare', 'both') 3. Calculate distance using Haversine formula
3. Create ride record in database Status: 'pending' Store pickup/dropoff coordinates
4. Send FCM push to nearby drivers "New ride request: $X.XX - Y miles" 2. Ride Created {ride_id, status, estimated_fare}
Waiting for 3. Driver accepts ride Driver assigned {driver_name, vehicle, eta}
4. Driver POST /api/v2/driver/rides/{id}/ accepts accept 5. Location updates Navigate Driver POST /api/driver/location to pickup (Live map) (every 10 seconds)
6. Driver arrives POST /api/v2/driver/rides/{id}/arrive Trip start Driver 7. Start ride Start Arrived POST /api/v2/driver/rides/{id}/ Trip start
8. In progress (live tracking continues) In Transit 9. Complete ride Complete (Live map) POST /api/v2/driver/rides/{id}/ Trip complete
10. Payment processed via Stripe Customer charged Driver receives 85% Platform receives $1-3 Rate & Earnings Review Updated
RIDESHARE API ENDPOINTS:
Customer: POST /api/rides/request - Request a ride GET /api/customer/rides - Get customer's rides GET /api/rides/{id}/track - Track ride in real-time POST /api/rides/{id}/cancel - Cancel ride request POST /api/rides/{id}/rate - Rate driver after ride
Driver: GET /api/v2/driver/rides/available - Get nearby ride requests POST /api/v2/driver/rides/{id}/accept - Accept a ride POST /api/v2/driver/rides/{id}/arrive - Mark arrival at pickup POST /api/v2/driver/rides/{id}/start - Start the ride POST /api/v2/driver/rides/{id}/complete - Complete the ride GET /api/erp/driver/{id}/rides - Get driver's ride history

```


7. API CONNECTION DETAILS

How iOS apps connect to the backend:

[illegible]

8. DATABASE SCHEMA

Core tables and relationships:

```

##### DATABASE SCHEMA #####
##### CUSTOMERS VENDORS #####
(Restaurants) ##### id (PK) id (PK) ##### email (unique) email (unique) ##### password_hash
password_hash ##### full_name business_name ##### phone phone ##### google_id address, city, state, zip ##### apple_id latitude, longitude #####
stripe_customer_id cuisine_type ##### created_at is_active ##### rating ##### stripe_account_id ##### DRIVERS ORDERS #####
id (PK) id (PK) ##### email (unique) customer_id (FK → customers) ##### password_hash vendor_id (FK → vendors) ##### first_name, last_name
driver_id (FK → drivers) ##### phone status (pending/confirmed/preparing/ ##### vehicle_type ready/picked-up/delivered/cancelled) #####
vehicle_color order_total ##### license_plate platform_fee ($1-3 tiered) ##### drivers_license (verified) delivery_fee ##### insurance
(verified) tax ##### background_check (verified) tip ##### is_online grand_total ##### current_lat, current_lng delivery_address ##### driver_type
(delivery/rideshare/ special_instructions ##### both) created_at ##### stripe_account_id estimated_delivery_time ##### RIDES (Rideshare)
MENU_ITEMS ##### id (PK) id (PK) ##### customer_id (FK → customers) vendor_id (FK → vendors) #####
driver_id (FK → drivers) name ##### status (pending/accepted/ description ##### in_progress/completed/ price ##### cancelled) category #####
pickup_lat, pickup_lng image_url ##### pickup_address is_available ##### dropoff_lat, dropoff_lng is_vegetarian ##### dropoff_address is_vegan #####
distance_miles is_gluten_free ##### fare_amount spice_level ##### platform_fee prep_time_minutes ##### driver_earnings ##### created_at #####
started_at ##### completed_at #####
##### ENTITY RELATIONSHIPS #####
CUSTOMERS ##### 1:N ##### ORDERS ##### N:1 ##### VENDORS ##### 1:N ##### RIDES ##### 1:N ##### MENU_ITEMS #####
DRIVERS ##### N:1 ##### ORDERS #####

```

9. PAYMENT FLOW (Stripe P2P)

How money flows through the system:

[illegible]

10. REAL-TIME COMMUNICATION

Push notifications and live tracking:

```

##### REAL-TIME COMMUNICATION #####
##### PUSH NOTIFICATIONS #####
(Firebase Cloud Messaging) ##### Customer App receives: ##### Order
confirmed by restaurant ##### Order being prepared ##### Driver assigned ##### Driver picked up order ##### Driver arriving soon #####
Order delivered ##### Promotions and deals ##### Driver App receives: ##### New delivery available nearby ##### New ride request nearby
##### Order ready for pickup ##### Customer message ##### Earnings update ##### Restaurant App receives: ##### New order received #####
Order cancelled ##### Driver arriving for pickup ##### Low inventory alerts #####
##### LIVE LOCATION TRACKING #####
##### Driver App: ##### 1. LocationManager requests GPS updates every 10 seconds ##### 2. POST
/api/driver/location ##### { driver_id, latitude, longitude, heading, speed } ##### 3. Backend stores in drivers table: current_lat,
current_lng ##### Customer App: ##### 1. Polls GET /api/erp/orders/{id}/driver-location every 5 seconds ##### 2. Updates map marker with
driver position ##### 3. Calculates and displays ETA #####
##### CUSTOMER APP DRIVER APP #####
##### Manager ##### ← ← ← (GPS) ##### Location ##### You ##### updates POST every #####
##### via API ##### 10 seconds #####
##### DEMO CREDENTIALS #####
##### Customer: demo@dollar.ai / DollarDemo2024! Driver: demodriver@dollar.ai / DollarDriver2024! Restaurant:
demobusiness@dollar.ai / DollarBiz2024! API BASE URL: https://api.dollar.ai/api API DOCS: https://api.dollar.ai/docs PLATFORM STATUS:
PRODUCTION READY

```